Spring Cloud Microservices Lab: Student Guide (12 Topics)

This guide explains twelve topics of a microservices system, one by one, from zero. Each topic starts with the problem it solves, then the idea, the tool used in our lab, the alternatives, and what you will see when you press the buttons in the lab console.

Start here: the big picture

Imagine a **shopping mall**. The old way of building software is one giant shop that sells everything under one roof. If its cash counter breaks, the whole shop stops. This is called a **monolith**.

The modern way is a mall of many small shops. Each shop does one job, has its own staff, and can be repaired or copied on its own. This is called **microservices**: many small programs, each doing one job, working together.

Small shops create new problems. The twelve topics are the twelve answers.

Problem in the mall	Topic	The solution
Something broke. Eight shops each keep their own notebook. How do we read the whole story?	1	One central records room (ELK)
How do we know a shop is healthy and how busy it is?	2	A dashboard on every shop (Actuator, Micrometer)
Who collects those numbers, keeps history and rings an alarm?	3	A clerk and a notice board (Prometheus, Grafana)
A customer wants shoes. Which shop, and at what address?	4	Two ways of finding shops (discovery patterns)
Shops open, close and move. Who keeps the directory current?	5	The mall receptionist (Eureka)
Three identical shoe shops exist. Who gets the next customer?	6	A fair way to choose (load balancing)

Problem in the mall	Topic	The solution
A shop closed suddenly. How does the directory find out?	7	Roll calls and sick notes (health checks)
Customers should use one entrance. Who sends each to the right shop?	8	The main entrance and its sorting desk (gateway routes)
Every visitor needs an ID check and a record. Should each shop do it?	9	Checks done once at the door (gateway filters)
One pushy customer blocks everyone else.	10	A ticket counter (rate limiting)
What if the shop we send you to is closed?	11	A guide who walks you to another one (retry)
Phone users and laptop users need different things.	12	A personal helper per type of visitor (BFF)

The topics build on each other. Topics 1 to 3 are about **watching** the system. Topics 4 to 7 are about **finding** services and surviving failures. Topics 8 to 12 are about the **front door**.

Words used everywhere

Word	Plain meaning
Service	A small program that does one job and can be called by other programs
Instance	One running copy of a service. "Three instances of product-service" means three copies running at once
Container / Docker	A container is a sealed box holding one program and everything it needs. Docker runs these boxes, so the lab behaves the same on every computer
Request / response	A request is a message "please give me this". The response is the answer
HTTP	The language web programs use to send requests and responses

Word	Plain meaning
Port	A numbered door on a computer. Each program listens at its own port
API	The list of requests a program understands, like a restaurant menu
JSON	A simple text format for data, for example `{"name":"Keyboard","price":89.99}`
Header	Extra information attached to a request or response, like a sticker on a parcel
Status code	A number in every response: 200 OK, 401 not logged in, 403 not allowed, 404 not found, 429 too many requests, 500 program error, 503 service unavailable
Latency	How long a request takes to be answered
Timeout	The longest we are willing to wait before giving up
Cache	A saved copy kept close by so we do not ask again every time. It can become out of date

How to read each topic

Every topic follows the same pattern, so you always know where you are.

1. **The problem:** why the topic exists.
2. **The idea:** the concept in plain words, often with an everyday picture.
3. **The tool and its alternatives:** what the lab uses, and what else could do the same job. The concept matters more than the tool. Tools change, concepts stay.
4. **Key words:** the vocabulary you need.
5. **In the lab:** what the buttons do and what you should see.
6. **Think about it:** questions to test yourself.

Topic 1: Centralized logging with ELK

When something fails in a system of many programs, each program has written its own diary, so finding one story means reading eight diaries on eight machines. Centralized logging sends every diary line to one searchable place.

The problem

A **log** is a line of text a program writes about what it did, like `10:15:02 ERROR could not reach database`. One program has one log file, and you open it. A modern system has 8 or 80 programs running in containers, and containers are deleted and replaced all the time, so their log files vanish with them.

Imagine a customer says "my order failed at 2:15". You would have to guess which program failed, log into each one, and read thousands of lines, if the files still exist.

The idea

Every program sends each log line to one central place the moment it is written. That place stores them, lets you search them in seconds, and survives when a container does not. A hospital works the same way: every department sends a copy of its notes to one records room, filed under the patient's file number.

The file number here is the **request id**, a short code created once at the entrance and copied into every log line of that request. Search the code and you see the whole story, in order, across all programs. This is also called a **correlation id**.

The tool: ELK

ELK is three tools working as a pipeline.

Tool	Job	Picture
Logstash	Receives log lines, cleans or tags them (for example marks every ERROR as "needs attention"), and passes them on	The postman
Elasticsearch	Stores the lines and searches the full text very fast	The storage cabinet
Kibana	A web page where you search, filter and chart the stored lines	The reading window

Why ELK? It is free, very widely used, and searches the full text of every line.

Honest downsides: ELK is heavy (about 2 GB of RAM in our lab), and storing every word gets expensive at large scale.

Alternatives

Tool	What to know
Grafana Loki	Lighter and cheaper. It indexes only labels such as service name, so text search is slower
Graylog	Similar to ELK, with a simpler setup
Splunk, Datadog	Paid products with more features
Cloud services (CloudWatch, Azure Monitor)	Built into each cloud

Key words

Word	Plain meaning
Log level	How serious a line is: DEBUG (tiny detail), INFO (normal), WARN (strange), ERROR (something failed)
Structured (JSON) log	A line written as labelled fields (level, service, time) so it can be filtered
Index	A "drawer" in Elasticsearch. In the lab there is one per day
Pipeline	Logstash's three steps: input (receive), filter (clean or tag), output (send on)
Stack trace	The detailed list of where in the program an error happened
KQL	Kibana's simple search language, for example `level : ERROR`
Buffer	A small waiting area in memory

In the lab

Button or area	What it does and what you should see
Browse products, Place an order	Make normal traffic. A new line appears in the black box each time
Cause an error	A planned failure. The line is red and shows status 500
Emit every log level	Writes one line of each severity so you can practise filtering
Live log viewer	Rows with time, level (INFO green, WARN amber, ERROR red), service, request id and message. Choose ERROR to keep only the red ones
Place an order and trace it	The key button. About 5 lines appear from 3 services (gateway, order-service, product-service), all with the same request id
Open in Kibana	Opens the professional tool with the same data
Kill Logstash	Press it, then browse products. Everything still works, because logging must never stop the shop. Press Start to bring Logstash back

Think about it

1. Why is `docker logs` not enough for a system with many services?
2. What does the request id let you do that a plain search for a word cannot?
3. What happens to log lines while Logstash is down? (Short outage: programs hold a few thousand lines in memory. Long outage: some are lost. Production systems add a durable queue such as Kafka.)
4. Why is full-text indexing both ELK's strength and its cost?

Topic 2: Actuator and Micrometer

A running program should be able to tell us, from outside, whether it is healthy and how busy it is, without us opening it up. Actuator provides the check-up pages, and Micrometer records the numbers.

The problem

A program is running. Is it healthy? How many requests has it served? How slow is it? You cannot look inside a running program, so it must answer from outside.

The idea

Think of a car dashboard: a warning lamp (is something wrong?), a fuel gauge, an odometer, a trip stopwatch. Or a nurse who stands at every program's door and measures temperature and heart rate. Each program exposes small web addresses that answer questions such as "how are you?" and "give me your numbers".

The tools

Actuator is a Spring Boot add-on that gives every program built-in check-up pages, such as `/actuator/health`. One line in the project's dependency list switches it on.

Micrometer is a library inside the program that records numbers (called **metrics**). It has three instruments:

Instrument	Behaviour	Car picture	Example
Counter	Only goes up	Odometer	Total orders so far
Gauge	Goes up and down	Fuel gauge	Items in stock right now
Timer	Records how long something takes and how often	Stopwatch	Product lookup time

Why not write your own `/health` page? You could, and many teams did. But then every team invents its own format and names, and no tool understands them. Actuator gives a standard that Docker, Kubernetes and monitoring tools already read.

Why Micrometer? It works like a universal plug, the same way SLF4J works for logs. You record a number once in your code, and a setting decides whether it goes to Prometheus, Datadog or CloudWatch.

Alternatives

Tool	What to know
OpenTelemetry	A newer, vendor-neutral standard for metrics, logs and traces. Growing fast
Dropwizard Metrics	An older Java library, replaced by Micrometer in Spring
Prometheus client libraries	The equivalent for other languages such as Python and Go
A hand-written health page	Works, but nobody else's tools understand it

Key words

Word	Plain meaning
Endpoint	One web address that answers one question, for example `/health`
Metric	A number describing behaviour: requests per second, memory used
Tag / label	Extra detail on a metric, such as `category = peripherals`, so numbers can be split by it
Health check	A quick test: is this program able to do its job?
Liveness	Is the process stuck or dead? This is different from health
HTTP 503	"Service unavailable": the program says it cannot serve now

In the lab

Button or area	What it does and what you should see
health	Shows `"status": "UP"` and a list of parts (disk space and others). UP means healthy

Button or area	What it does and what you should see
prometheus text	All the program's numbers in the text format Prometheus reads. Type `lab_` in the filter to see only numbers made by this lab
view product 1, 2, 3	Makes the counter go up. After ten clicks press Refresh meters: a gauge, a timer, and a counter split by instance. Each of the three copies counts only what it served itself
Change log level, then Apply	Switch from INFO to DEBUG and call log-demo again. The behaviour changes with no restart. Put it back to INFO at the end
Break product-2	`/health` shows DOWN (HTTP 503) but `/health/liveness` stays UP. The program is alive but not well. Press Heal product-2 to fix it

Think about it

1. Is it safe to show every Actuator page to everyone? (No. Some show configuration. Real systems restrict access.)
2. What is the difference between a counter and a gauge? Give one example of each from a shop.
3. Why can a program be alive but not healthy?
4. Why do the three product copies show different numbers, and who adds them up? (Topic 3.)

Topic 3: Prometheus and Grafana

Numbers from one program are only useful if something collects them regularly, keeps the history, draws charts and rings an alarm. Prometheus collects and stores, Grafana draws, and alert rules watch while nobody is looking.

The problem

Topic 2 let each program report its numbers when asked. That leaves four questions:

1. Who asks? Nobody will visit 8 programs by hand.
2. Who remembers yesterday's numbers? A program only knows its current value.

3. Who adds up the 3 product copies to get a total?
4. Who notices at 3 a.m. when something is wrong?

The idea

Think of a school. A **clerk** walks to every classroom every 5 seconds and writes numbers in a big register. That clerk is **Prometheus**. A notice board in the corridor shows charts of the register: that is **Grafana**. A fire bell rings if a classroom is suddenly empty: that is an **alert**.

The tools

Prometheus is the collector and the database.

- Every 5 seconds it visits each program's metrics page and copies the numbers. One visit is a **scrape**, and each program visited is a **target**.
- It stores every number with its timestamp. A number measured again and again over time is a **time series**.
- You ask it questions in its own language, **PromQL**, such as "requests per second, by service".
- It checks **alert rules**, such as "if a target has been down for 10 seconds, raise an alarm".

Grafana stores no data. It asks Prometheus questions and draws the answers as charts. A screen of charts is a **dashboard**, and each chart is a **panel**.

The key design idea: pull, not push

Prometheus **pulls**: it goes to the programs, and the programs do not send their numbers. Why?

- If a program dies, Prometheus gets no answer, and that silence is itself information. With push, a dead program just stops sending, and you cannot tell "dead" from "quiet".
- Adding a new program needs no change inside it.
- You can open a program's metrics page in a browser and see exactly what Prometheus sees.

The downside: pull works poorly for very short jobs that finish between two scrapes. Prometheus has a helper called Pushgateway for those.

Why these tools, and alternatives

Both are free and open source. Prometheus is the standard in the Kubernetes world, and Spring Boot supports it with one dependency through Micrometer.

Alternative	What to know
Datadog, New Relic, Dynatrace	Paid, all-in-one, nothing to run yourself. Costs grow with size
AWS CloudWatch, Azure Monitor, Google Cloud Monitoring	Built into each cloud, tied to that cloud
InfluxDB, VictoriaMetrics	Other time-series databases. VictoriaMetrics is Prometheus-compatible and handles larger volumes
Thanos, Mimir	Add-ons giving Prometheus long-term storage and scale

Limit to remember: one Prometheus keeps its data on one machine, so very large or long-term setups need the add-ons above. It is for numbers, not for logs (Topic 1).

Key words

Word	Plain meaning
Rate	How fast a counter grows per second. "Requests per second now" is more useful than "total so far"
Label	Extra detail on a number, such as service or instance. Labels let you split or add up numbers
Percentile (p95)	"95 out of 100 requests were faster than this." It shows the slow end that an average hides
Histogram	Timings sorted into ranges (buckets), used to calculate percentiles
Pending / firing	An alert that has started to be true / one that has been true long enough to ring

Why p95 matters: if 99 requests take 0.1 seconds and 1 takes 10 seconds, the average is about 0.2 seconds and looks fine, but one user in a hundred waited 10 seconds.

In the lab

Button or area	What it does and what you should see
Targets table	About 8 rows, all green **up**, with a "last scrape" time that keeps changing
Start traffic	Sends 10 requests per second from your browser so the charts have data. An idle system gives flat charts
PromQL playground	Pick a ready-made question. "Requests per second, by service" shows how busy each part is. "Requests per product instance" shows three lines of nearly equal height, proof that work is shared. "Latency p95" shows how slow the slowest 5% are
Edit `[30s]` to `[5m]`	Changes the measuring window. Short windows react fast but are noisy. Long windows are smooth but slow
Freeze on product-service-2	The program stops answering without dying. Its row turns red within 5 to 10 seconds, and about 10 seconds later the **InstanceDown** alert shows as firing. Unfreeze reverses it
Grafana panels	The same data as four charts. Each chart is just a PromQL question underneath

Think about it

1. Why does Prometheus pull instead of waiting for programs to push?
2. Why is p95 more honest than an average?
3. What is the difference between logs and metrics? (Metrics are cheap numbers over time, good for "is something wrong?". Logs are detailed text, good for "what exactly happened?". You usually need both.)
4. Why is it a bad idea to put a user id in a label? (It creates millions of series and can exhaust Prometheus's memory.)

Topic 4: Service discovery patterns

Programs start, stop and move all the time, so a caller cannot remember a fixed address. It must look up the current addresses by name, and the real question is who does the looking up and choosing: the caller itself (client-side) or a middle program (server-side).

This topic introduces one idea and no new tool. Topic 5 (Eureka) is the tool that makes the idea work.

The problem

Suppose order-service needs to call product-service. It needs an address like `http://172.18.0.7:8080`. A fixed address is a bad idea for three reasons:

1. **Addresses change.** Each time a container restarts, it may get a new address.
2. **There are several copies.** Which of the three product-service copies should the address point to?
3. **Copies come and go.** Today three, tomorrow five, and one may crash at noon.

If the address is written in the code, it breaks the first time anything restarts. Programs must find each other **by name** (`product-service`) and look up the current addresses when they need them. This is **service discovery**. The directory holding the names and addresses is the **service registry**.

The idea: who chooses?

Imagine finding a shoe shop in a huge mall.

- **Client-side discovery:** you read the directory board yourself, see three shoe shops, choose shop B and walk there. *You* decide.
- **Server-side discovery:** you go to the information desk and say "I need shoes". The person looks at the list, chooses a shop and sends you. *The desk* decides, and you never see the directory.

In our lab:

- **Client-side:** order-service calling product-service. Order-service has the directory library inside it.
- **Server-side:** the browser calling the gateway. The browser knows only the gateway's address, and the gateway does the lookup.

Trade-offs

	Client-side	Server-side
Stops on the way	One fewer, so slightly faster	One extra stop (the router)
Caller complexity	The caller needs discovery code and a library	The caller is very simple: one address
Good for	Programs inside the system, written by your own team	Outside callers such as browsers and phones
Weak point	Every caller keeps its own copy of the directory, which can be a few seconds stale	The middle program must never fail, and it can become a bottleneck

Neither is better. Real systems use both: outsiders come through a gateway, and programs inside talk directly.

A point students often enjoy: the gateway is server-side *for the browser*, but inside the gateway the lookup uses the same client-side library. The pattern depends on **who the caller is**, not on the technology.

Tools and alternatives

Spring Cloud with Eureka is the classic client-side setup. It is good for learning because every piece is visible. Many companies now do it differently.

Approach	What to know
Kubernetes Services and DNS	Server-side. You call a name and Kubernetes finds and chooses a copy. The most common setup today
Consul (HashiCorp)	A registry like Eureka that works with any language
ZooKeeper, etcd	Lower-level stores often used for the same job
Cloud load balancers (AWS ELB, Cloud Map)	Server-side, managed by the cloud

Approach	What to know
Service mesh (Istio, Linkerd)	A helper next to each service does the lookup, so your code needs no discovery library

Will you use Eureka in a job? Possibly in older Spring systems. The concept transfers to every row of this table.

Key words

Word	Plain meaning
Service discovery	Finding the current address of a service by its name
Service registry	The directory that stores names and addresses
Logical name	A name like `product-service` that stands for "any live copy of it"
Hop	One stop on the way of a request
Cache	A saved copy of the directory kept by each caller. It can be a few seconds out of date
Load balancer	The part that picks one copy from several (Topic 6)

In the lab

Button or area	What it does and what you should see
Client-side: Send 9 calls	You press once, and order-service makes 9 calls. You see 9 coloured squares in a repeating three-colour pattern, bars of 3 / 3 / 3, and the sentence "Browser made 1 request; order-service made 9 requests"
Server-side: Send 9 calls	The browser makes 9 requests to one address (the gateway). The squares and bars look the same, but the sentence says "Browser made 9 requests to ONE address"

Button or area	What it does and what you should see
Three copies of the phone book	Three boxes list what Eureka, the gateway and order-service each know. Normally they agree. After a failure they can disagree for a few seconds

The lesson of the two experiments: the result looks the same, but **the work happened in a different place**. Watch the dots on the live map to see where.

Think about it

1. Why is a hard-coded address a bad idea in a system of containers?
2. In the lab, which experiment is client-side and which is server-side? How can you tell?
3. Why does every caller keep its own copy of the directory, and what is the cost?
4. Is a gateway a single point of failure? How do production systems reduce that risk?

Topic 5: Eureka, the service registry

Eureka is the directory from Topic 4, kept up to date by the programs themselves: they register when they start, send a heartbeat every few seconds, and say goodbye when they stop. If the heartbeats go silent, Eureka removes the entry, unless too many go silent at once, in which case it stops removing anyone (self-preservation).

The problem

A directory that is written once goes wrong within minutes. Copies are added, restarted and killed, and nobody can update the list by hand. Eureka's answer is that **the programs keep the list up to date themselves**.

What Eureka is

Eureka was created by Netflix, and Spring Cloud wraps it. It has two sides:

- **Eureka server:** the directory (`eureka-server` on the lab map).
- **Eureka client:** a small piece of code inside every other program that talks to the server.

The four things a program does

Picture a hospital receptionist with a notebook. Doctors sign in, tap a bell regularly to say "I'm here", and say goodbye when they leave.

Step	Technical name	Plain meaning	Real default	Lab setting
1	**Register**	On start: "I'm product-service, at this address"		
2	**Heartbeat**	Every few seconds: "I'm still alive"	every 30 s	every 3 s
3	**Cancel (deregister)**	On a polite stop: "I'm leaving", removed at once		
4	**Evict**	The server removes an entry whose heartbeats stopped	after 90 s of silence	after 6 s

The promise to keep sending heartbeats is a **lease**. If it is not renewed in time, the lease expires and the entry is evicted. The lab uses times about 10 times shorter so a lesson takes seconds. In real systems the defaults are 30 and 90 seconds.

Callers also keep a copy of the directory and refresh it periodically (30 seconds by default, 5 in the lab). That is the stale copy from Topic 4.

Self-preservation

The rule: Eureka counts the heartbeats it receives each minute. If fewer than **85%** of the expected number arrive, it assumes the problem is itself or the network, not the programs, and **stops evicting anyone** until the heartbeats return.

The story: a storm breaks the bell wires in half of the hospital. The receptionist suddenly hears far fewer bells. She thinks: "It is very unlikely that half my doctors vanished together. More likely my own wires are broken." So she stops crossing anyone off until the bells work again.

Why it matters: suppose a network fault cuts Eureka off from half the services. They are fine, but Eureka cannot hear them. Without the rule it would delete half the directory, and a small network problem would become a total outage.

The trade-off in one sentence: it prefers "a few wrong names" over "an empty list". The cost is that dead programs may be listed as UP, so callers must expect failures and use timeouts and retries (Topics 7 and 11).

Why Eureka, and alternatives

It is free, simple to see working, and Spring supports it with very little setup. Be honest about its place today:

Alternative	What to know
Kubernetes Services	Does discovery for you, built in. The most common choice today, so many teams no longer need Eureka
Consul (HashiCorp)	A registry like Eureka that works with any language and has stronger health checking
ZooKeeper, etcd	Lower-level stores that can hold a registry. etcd is what Kubernetes itself uses
Cloud services (AWS Cloud Map)	A managed registry, nothing to run yourself
Service mesh (Istio, Linkerd)	Moves discovery out of your code entirely

Key words

Word	Plain meaning
Registry	The list of which programs exist and where
Register	A program announces its name and address when it starts
Heartbeat	A small regular message meaning "I'm alive"
Lease	The promise to keep sending heartbeats
Evict	The server removes an entry whose lease expired
Graceful stop	A polite stop: the program says goodbye (Docker's signal is SIGTERM)
Kill / crash	A sudden stop with no goodbye, like pulling the power plug (SIGKILL)
Freeze	The program is still there but does not respond, like a hung computer

Word	Plain meaning
Self-preservation	If too many heartbeats vanish at once, stop evicting
Network partition	A network break so some programs cannot reach others, though all are fine

In the lab

Button or area	What it does and what you should see
Live registry table	7 rows, each with a green UP badge. The "Since last heartbeat" bar fills up to about 3 seconds, then resets. That is the heartbeat made visible
Stop (graceful) on product-service-3	The row disappears within a few seconds, because the program told Eureka it was leaving. A polite goodbye is noticed at once
Heal	Brings everything back. Wait about 40 seconds for the program to start and register again
Kill (crash) on product-service-3	The row **stays**, its bar fills and turns red, and after about 12 to 15 seconds it disappears. Eureka did not know until the silence lasted long enough
Timeline	Records each event with how long Eureka took to notice, for example "noticed 15.1s after you pressed kill"
Self-preservation meter	The green bar is the heartbeats received in the last minute, and the red mark is the danger line. It only changes during the drill below
GET /eureka/apps/PRODUCT-SERVICE	Shows the raw directory data, to prove it is only text

Optional drill (about 6 minutes): set `EUREKA_LEASE_EXPIRATION=90` in `.env`, run `docker compose up -d`, then freeze four services (the three product services and order-service). Freezing two or three is not enough, because this Eureka version counts each heartbeat

about twice. After about 2 minutes the badge turns red (SELF-PRESERVATION ACTIVE) and the frozen services still show UP. Unfreeze them and set the value back to `6`.

Think about it

1. Why does a crash take about 15 seconds to notice when the lab lease is only 6? (Eureka stores each lease as "renewal time plus lease length", which nearly doubles the wait.)
2. What is the difference between a graceful stop and a crash from Eureka's point of view?
3. Is self-preservation a bug? Argue both sides.
4. A heartbeat proves the program can send a message. Does it prove the program can do its job? (This leads to Topic 7.)
5. Eureka uses push (programs report). Prometheus uses pull (Topic 3). What does each approach detect well?

Topic 6: Load balancing with Spring Cloud LoadBalancer

When three identical copies of a program can answer, something must decide which copy gets the next request, fairly and automatically. Round-robin takes turns, random rolls a die, and Spring Cloud LoadBalancer replaced the retired Netflix Ribbon.

The problem

There are three identical product-service copies and a request arrives. If the choice is always copy 1, then copy 1 is overloaded while 2 and 3 sit idle, and having three copies gained nothing. Sharing work between copies is **load balancing**. "Load" means the work, and "balancing" means sharing it.

The two strategies in the lab

Picture a supermarket with three identical cash counters and a staff member at the entrance sending customers to them.

Strategy	How it works	Strength	Weakness
Round-robin	Counter 1, then 2, then 3, then 1 again	Perfectly even and predictable	Needs a counter that remembers whose turn it is

Strategy	How it works	Strength	Weakness
Random	Every customer rolls a die	Needs no memory, so it is cheap with thousands of requests at once	Lumpy in the short run: three in a row may go to copy 2. Fair only over many requests

Both ignore how busy or fast a copy is. They work well when requests are similar and copies are identical.

Other strategies exist, though the lab does not demonstrate them: **least connections** (send to the copy with the fewest requests in progress, better when requests vary a lot), **weighted** (a bigger machine gets a bigger share), **hash-based or sticky** (the same user always goes to the same copy), and **zone-aware** (prefer copies in the same data centre).

Ribbon and its replacement

- **Ribbon** was the original load balancer from Netflix, used with Eureka. Netflix stopped developing it, and it was removed from Spring Cloud in version 2020.0.
- **Spring Cloud LoadBalancer** is its replacement, made by the Spring team. It asks the registry (Eureka) for the live copies of a service name, keeps a short-lived list (5 seconds in the lab, called the **cache TTL**), and picks one with the chosen strategy.

If you find an old tutorial about Ribbon, now you know why it is out of date. In the lab, open `SwitchableLoadBalancer.java` in the Source tab. It is only about 15 lines, which shows how small the idea is.

Alternatives

It is built into Spring, so a Spring program gets client-side load balancing with almost no setup, using the same Eureka list as Topic 5.

Alternative	What to know
Nginx, HAProxy	Classic server-side load balancers in front of services. Very common and fast
Envoy	A modern proxy used in service meshes such as Istio
Cloud load balancers (AWS ELB, Google, Azure)	Managed, nothing to run yourself

Alternative	What to know
Kubernetes Services	Built-in load balancing across copies

Most of these are server-side. Spring Cloud LoadBalancer is client-side (Topic 4).

Key words

Word	Plain meaning
Load	The amount of work (requests)
Load balancer	The part that picks one copy for each request
Strategy / algorithm	The rule used to pick
Distribution	How requests ended up split between copies
Cache TTL	How long the saved list of copies is kept before asking again

In the lab

Button or area	What it does and what you should see
Round-robin / Random	Switches the strategy **while the system runs**, with no restart. A line confirms it, for example `gateway: round-robin | order-service: round-robin`
Number of requests	9, or 300 (fast)
Send via the gateway	The gateway picks the copies. You see coloured squares (one colour per copy) and three bars
Send via order-service	The same idea, but a different program is choosing (Topic 4)

What you should see:

- **Round-robin, 9 requests:** a repeating colour pattern and bars of exactly 3 / 3 / 3.
- **Random, 9 requests:** any order and uneven bars such as 4 / 1 / 4. Press again for a different split.

- **Random, 300 requests:** each bar close to 33%.

Set the strategy back to Round-robin when you finish.

Think about it

1. Why would anyone choose random when round-robin is more even?
2. Why not always choose the least busy copy? What would that cost?
3. What happens if one copy is slower than the others? Do these two strategies notice?
4. What happens if a dead copy is still on the list? (Topics 7 and 11.)
5. What is the difference between Spring Cloud LoadBalancer and Nginx?

Topic 7: Health checks and automatic removal

A directory is only useful if it quickly stops pointing at broken programs. But "broken" comes in four forms, each noticed in a different way and at a different speed, and a heartbeat alone does not prove a program can do its job.

The problem

Topic 5 showed that Eureka removes a program only after its heartbeat goes silent. The harder question is: how does the system find out about each kind of failure, and how many customers are hurt before it does?

The four failures

Think of a hospital duty roster that says which doctors are on duty.

Failure	Picture	Lab button	How the system finds out	Speed
Unwell but present	A doctor with a fever, still at her desk, tells reception "I'm unwell"	Health DOWN	The program reports its own bad health, and its status in Eureka flips to DOWN	About 4 to 10 s
Polite goodbye	A doctor says "I'm leaving"	Stop (graceful)	The program deregisters itself	A few seconds
Crash	A doctor faints, no goodbye	Kill (crash)	Eureka waits until the heartbeats stop	About 12 to 15 s

Failure	Picture	Lab button	How the system finds out	Speed
Frozen	A doctor is in her room but never answers the phone	Freeze	Eureka eventually notices, but until then callers wait on a program that accepts calls and says nothing	Slowest, and most damaging

Frozen is the worst. A crashed program refuses at once, so the caller fails fast. A frozen program accepts the call and goes silent, so the caller waits the full timeout. Slow failures hurt more than fast ones, which is why you always set timeouts.

Three layers of protection

1. **Heartbeat** (Topic 5): catches programs that died.
2. **Health status:** catches programs that are alive but sick. A program that lost its database still sends heartbeats, so Eureka would call it fine. A health check lets it say "take me out of service".
3. **Caller protection:** timeouts and retries (Topic 11) cover the gap before layers 1 and 2 react. There is always a gap.

The key sentence: **alive is not the same as healthy.**

Liveness vs readiness

Kubernetes made these terms standard:

- **Liveness:** is the process stuck or dead? Should it be restarted?
- **Readiness:** can it take requests right now? A program that is starting up, or has lost its database, is alive but not ready. It should be taken out of rotation, not killed.

The Health DOWN button in the lab is the readiness idea.

Alternatives

The lab uses Spring's health endpoint, Eureka status and gateway retry. Others:

Alternative	What to know
Kubernetes probes (liveness, readiness, startup)	The most common today. Kubernetes checks each pod and removes it from the Service when it is not ready
Load balancer health checks (Nginx, HAProxy, AWS ELB)	The balancer actively calls each copy on a timer
Consul checks	The registry calls the program instead of waiting for heartbeats
Circuit breakers (Resilience4j)	The caller stops calling a failing service for a while and fails fast
Envoy outlier detection	A proxy ejects copies that return too many errors

A key difference: Eureka is **push** (programs report), while Kubernetes and Consul checks are often **pull** (the system probes). Pull can catch a frozen program sooner.

Key words

Word	Plain meaning
Health check	A test of whether a program can do its job right now
Probe	A test request sent on a timer to watch behaviour
Deregistration	Removal of an instance from the registry
Retry	Sending a failed request again, to another copy
Timeout	The longest we wait for an answer
Stale	Out of date
Blast radius	How many users one failure hurts

In the lab

Start all three probes first. Every half second, three kinds of caller send one request each, and each request is one square. The colour shows who answered: blue for product-1, orange for product-2, magenta for product-3. Amber means slow, and red means failed.

Row	Caller
1	Through the gateway **with** Retry
2	Through the gateway **without** Retry
3	order-service calling directly (client-side, no retry)

Drill	What you should see
Health DOWN	Within about 4 to 10 seconds no orange squares appear in any row, and no red either. The timeline says `status UP -> DOWN`. The system routed around the sick program. Press Health UP and orange returns
Kill (crash)	Row 1 stays mostly coloured, with a few amber squares. Rows 2 and 3 show red squares for about 12 to 15 seconds, then stop when the dead copy is removed. Those red squares are customers who would have seen an error
Freeze	Amber squares first (answers taking seconds), then red. Press Unfreeze
Heal	Restarts anything that is down. Wait about 40 seconds for it to register again

The second experiment shows three boxes with the registry as Eureka, the gateway and order-service each see it. They disagree for a few seconds after a failure, and that disagreement explains every red square.

Think about it

1. Why do the red squares stop by themselves?
2. Why does Row 1 hardly show any red? Does retry remove failures completely?

3. Why is a frozen program worse than a crashed one?
4. What should a health check include? What goes wrong if every service marks itself DOWN when a shared database is slow?
5. Why is restarting a sick program not always the right fix?

Topic 8: API Gateway routes and predicates

Visitors should know only one address. The API Gateway is that single front door: it reads each request and uses a table of rules (routes) to decide which internal program handles it. The first matching rule wins, and if none matches the answer is 404.

Topics 1 to 7 were about watching and finding services. From here on we build the front door.

The problem

You have about ten internal programs. If the browser talks to each one directly:

1. The browser must know every internal address, and those change (Topic 4).
2. Every program must handle login, logging and speed limits itself, ten times over.
3. You expose all your internals to the whole internet.

The fix is **one door** that everyone uses.

The idea: a sorting room

A post office receives all letters at one door. In the sorting room each letter is read, and simple rules decide which shelf and delivery van it goes to. In Spring Cloud Gateway, a **route** has four parts:

Part	Meaning	Post office picture
id	The route's name	The shelf label
Predicates	Conditions the request must meet	The sorting rules (address, stamp, weight)
Filters	Changes made to the request or answer (Topic 9)	Stamping or repackaging
uri	Destination	The delivery van

Three rules to remember:

1. A request must meet **all** predicates of a route for that route to apply.
2. Routes are checked in order. A lower `order` number is checked first, and **the first match wins**.
3. If no route matches, the answer is **404**.

What a predicate can check

- **Path:** the address, such as `/api/products/**`
- **Method:** GET, POST and so on
- **Query parameter:** such as `?preview=true`
- **Cookie:** a small value the browser sends back
- **Header:** such as `X-Client-Type: mobile`
- **Time window:** the route only works between two times

Spring Cloud Gateway has more predicates than these. Check its documentation for the full list.

The destination: `lb://`

In the route table you will see `lb://product-service`. This is not a real address. `lb` means "ask the load balancer (Topic 6) for a live copy of this service name, using the Eureka list (Topic 5)". Topics 4 to 6 come together here.

YAML vs Java DSL

The same route can be written two ways:

- **YAML:** a plain text configuration file. Easy to read, and an operations person can change it without programming.
- **Java DSL:** the same route written as Java code. Better when the rule needs real logic.

Both end up in the same table. The lab labels each row green **YAML** or blue **Java DSL**.

Why Spring Cloud Gateway, and alternatives

It fits Spring systems, uses the same Eureka and load balancer, and is easy to extend with Java. It is built on a non-blocking engine (Netty), so it handles many connections with few threads.

Alternative	What to know
Kong	Popular open-source gateway with many plugins
Nginx	Very fast reverse proxy, often used as a simple gateway
Envoy	Modern proxy used in service meshes
Traefik	Discovers services automatically, popular with Docker and Kubernetes
AWS API Gateway, Azure API Management, Google Apigee	Managed gateways
Zuul	The old Netflix gateway that Spring Cloud Gateway replaced, like Ribbon in Topic 6

Key words

Word	Plain meaning
API Gateway	The single entrance in front of all services
Route	One rule: conditions, changes, destination
Predicate	A condition that must be true for the route to apply
Path	The part of the address after the host, like `/api/products`
HTTP method	The verb: GET reads, POST creates
Query parameter	Extra text after `?` in an address
Order	The number deciding which route is checked first
404	"No such thing found"

In the lab

The live route table is the real list inside the running gateway, about 15 rows, with columns for order, id, where it is defined, predicates, filters and uri. Do not read every row. Point at `lb://product-service`.

In the request experiment, press each preset and read the result: HTTP code, matched route, where it was forwarded, and "Why this route". Try to predict the result before you press.

Preset	What you should see	The lesson
Path + Method	200, route `y-products`	The basic route
Query ? preview=true	A different route, `y-products-preview`	An extra condition wins an earlier route
Cookie: beta user	Route `y-products-beta-cookie` and an extra `X-Beta` header	Routing by cookie
Wrong method POST	404 with "why no route?"	The route accepts only GET
Time window	200 via `y-promo-window`	A route can be active only for a period
Expired window	404	The period has ended
Java DSL route	Route `dsl-whoami-envelope`, body wrapped in `_gateway` and `original`	The same table, written in Java
Header picks the BFF	Route `dsl-bff-by-header-mobile`	Needs you signed in as alice. Topic 12 builds on it
No route	404	No rule matched

You can also type your own path and press Send.

Think about it

1. Why not let the browser call each service directly?
2. What happens if two routes both match?

3. Why does a POST to a GET-only route return 404 and not 405 Method Not Allowed? (No route matched, so the request never reached a service that could say "wrong method".)
4. When would you choose YAML and when Java?
5. Should business rules live in the gateway? (No. Keep it for routing, security and limits.)

Topic 9: Gateway filters

Some jobs must happen to every request: check who the caller is, write a record, stamp an id. Doing them once at the gateway is safer and simpler than repeating them in every service. Global filters run for every route, and per-route filters run only where attached.

Topic 8 decided where a request goes. This topic is about what happens to it on the way.

The problem

If each of ten services does login checks and logging itself, you write the same code ten times, and one service will eventually do it wrongly. Do these jobs **once, at the door**.

What a filter is

A **filter** is a small piece of code that can look at and change a request on its way in, and the answer on its way out. Think of an airport:

- **Global filter:** the security check everyone passes, whatever the airline.
- **Per-route filter:** an extra check at one gate only, such as a passport check for one country.

	Global filter	Per-route filter
Applies to	Every route automatically	Only routes where it is listed
Good for	Request id, logging, identifying the caller	"Only logged-in users here", retry, rate limit, rewriting

Filters form a **chain**. The request passes through them in order, and the answer comes back through them in reverse. The lab shows the chain as RequestId, Logging, AuthHeaderInjection, RequireAuth or Retry, then the service.

The three global filters in the lab

1. **RequestId:** gives every request a short code. This is the code that followed an order through three services in Topic 1.
2. **Logging:** writes a line when a request arrives and when the answer leaves, with route, status and duration.
3. **AuthHeaderInjection:** reads the visitor's token, works out who they are, and writes that identity into headers for the services behind the gateway.

The per-route filters you will see are **RequireAuth** (an orange chip) and **Retry** (Topic 11).

The key idea: remove, then write

The gateway first **throws away** any identity headers the visitor sent, then **writes its own**. The services behind it receive `X-User-Id`, `X-User-Roles`, `X-User-Tier` and `X-Auth-Source`, and they can trust them because only the gateway can set them.

If the gateway did not remove the visitor's version, anyone could type `X-User-Id: admin` and be believed. A second benefit: the secret token stays at the gateway, and `Authorization` arrives at the service as absent.

Authentication vs authorisation

Word	Question it answers	In the lab
Authentication	Who are you?	The global filter reads the token
Authorisation	What are you allowed to do?	The per-route RequireAuth checks the role

Result	Meaning
401	Not logged in ("who are you?")
403	Logged in but not allowed ("I know you, but no")

Response rewriting

A filter can also change the **answer**. In the lab the gateway masks prices or wraps the answer in an envelope, and the product service is untouched. The cost: to change the answer the gateway must read all of it first, which uses memory and time. That is fine for small answers and bad for large ones or streams.

Alternatives

The lab's tokens (like `token-alice-123`) are a **teaching shortcut**. Real systems use signed tokens such as JWT from a login provider, and the gateway verifies the signature.

Alternative	What to know
Kong, Nginx, Envoy	The same idea through plugins or filters
AWS API Gateway authorizers, Azure API Management policies	Managed versions of the same checks
Spring Security with OAuth2 and JWT	How real systems verify identity
Keycloak, Auth0, Okta	Dedicated login providers that issue the tokens
Service mesh (Istio)	Moves checks like these next to each service

Key words

Word	Plain meaning
Filter	Code the gateway runs on requests or answers
Chain	The filters in the order they run
Token / bearer token	A secret code sent with a request to prove identity
Role	A label such as USER or ADMIN that decides permissions
Header injection	The gateway adds trusted headers to the request
Spoofed / forged header	A fake header sent by a visitor

Word	Plain meaning
Response rewriting	Changing the answer before the visitor sees it

In the lab

Do this	You should see
`/api/products/echo` as **anonymous**, Send	200. The service received `X-User-Id = anonymous` and `X-Auth-Source = gateway-anonymous`. The browser sent neither, so the gateway added them
Change to **alice**, Send	`X-User-Id = alice`, `X-User-Roles = USER`, `X-User-Tier = standard`, `X-Auth-Source = gateway`, and `Authorization` absent
Tick **forge X-User-Id: admin**, Send	"Browser sent" says admin, but "received" still says alice. The gateway threw away the fake
`/api/orders` as **anonymous**	401 (red)
`/api/orders` as **carol**	403
`/api/orders` as **alice**	200 with a list of orders, and an orange `RequireAuth(USER)` chip
Mask prices	The left box shows real prices like `89.99`, the right box shows `***masked***`
Wrap in an envelope	The right box shows the original data inside a new `_gateway` wrapper

The last experiment links to the log viewer from Topic 1 and the gateway log in Kibana. Gateway lines show route, status and duration for every request.

Think about it

1. Why must the gateway remove the headers the visitor sent?
2. What is the difference between 401 and 403?

3. Could someone skip the gateway and call a service directly with a fake header? What would stop them in a real system? (Network rules, or services verifying the signed token themselves.)
4. Why is response rewriting not free?
5. Does filter order matter? Give an example.

Topic 10: Rate limiting with Redis

One noisy or buggy caller can use up all the capacity, so the gateway limits how fast each caller may go. It uses a token bucket per caller, with the counters kept in Redis so that every gateway copy shares them.

Topic 9 checked who the caller is. This topic limits how much each caller may do.

The problem

One caller can starve everyone else. It might be an attacker, a scraper, or just a buggy program stuck in a loop sending 1,000 requests per second. Often it is not an attack at all, only a bug. The fix is a rule: each caller may go at most this fast. The gateway is the natural place, because every request passes through it and a refused request never reaches the services.

The token bucket

Picture a stadium gate with a bucket of tickets.

- The bucket holds at most **5** tickets (**burst capacity**).
- **2** new tickets drop in every second (**replenish rate**).
- Each request takes one ticket.
- No ticket means the guard says "wait": HTTP **429 Too Many Requests**.

Number	Name	What it controls
2 per second	Replenish rate	The long-run allowed speed
5	Burst capacity	How many requests may arrive at once

A calm visitor finds a full bucket and can send 5 requests at once. A caller who keeps going faster than 2 per second runs dry and is refused.

Plan in the lab	Replenish	Bucket
Standard	2 per second	5
Premium	10 per second	20

Why Redis?

Each caller has a bucket, and the bucket changes on every request. There may be several gateway copies, and a caller could hit different ones. If each gateway kept its own count, a caller could get N times the limit. So the counters live in **one shared place**: **Redis**, a very fast database that keeps data in memory.

In the lab, Redis holds two keys per bucket: the tokens left and the last-update timestamp.

Who owns a bucket?

The rule deciding "whose bucket is this?" is the **key resolver**. It could be the user, a client id, or an IP address. The lab lets you pick the bucket owner: this browser tab, alice or bob. Each gets a separate bucket, so one person being limited does not affect another.

Fail open vs fail closed

What should the gateway do if Redis dies?

Choice	Result
Fail closed	Refuse everyone. Safe from abuse, but the site is down
Fail open	Let everyone in. The site stays up, but unprotected

Spring's limiter **fails open**. In the lab, with Redis stopped, nothing is rejected, but each request becomes about 1 second slower while the gateway waits for Redis to time out. There is no single right answer: ask yourself which you would choose for a bank and which for a blog.

Alternatives

Alternative	What to know
Nginx `limit_req`	Simple and very fast, but counts are per Nginx server unless you add shared storage
Kong, Envoy, cloud gateways	Rate limiting as a built-in plugin or setting
Resilience4j RateLimiter	A library inside one program, protecting that service rather than the entrance
Bucket4j	A Java library for token buckets, can use Redis or other stores
Other algorithms	Fixed window (simplest, but allows double bursts at window edges), sliding window (smoother, costs more), leaky bucket (forces a steady output)

Key words

Word	Plain meaning
Rate limiting	Restricting how many requests a caller may make per second
Token bucket	The ticket-bucket method
Replenish rate	Tickets added per second
Burst capacity	Bucket size
429	"Too many requests, slow down"
Redis	A very fast in-memory database
Key resolver / bucket owner	The rule deciding whose bucket a request uses
Fail open / closed	When the safety system breaks: allow everyone / block everyone

In the lab

Do this	You should see
Standard plan, **Burst of 10**	About 5 green squares and 5 amber ones. Counter: `5 allowed 5 rejected (429)`. The bucket drawing is empty
Wait 2 seconds, **Send 1** a few times	One or two green (tokens came back), then amber again
Steady 4 per second for 10 s	After the first few, about half amber, since you ask for 4 and the plan gives 2
Premium plan, Clear, **Burst of 10**	All green, since the bucket holds 20
Look inside Redis	A table of the real keys. Change the bucket owner and burst again: new keys appear, one pair per owner
Stop (graceful) next to redis, then **Burst of 10**	Grey squares (the limiter failed open), nothing rejected, and each answer about a second slower. Press **Start** to repair it

In the lab the plan can be chosen by a header. That is a teaching shortcut: in a real system the plan comes from a verified token, not a header the visitor can type.

Think about it

1. Why return 429 instead of making the caller wait?
2. Why Redis and not a normal database?
3. Why does a bucket allow bursts, and when is that good or bad?
4. What could a determined attacker do to get around per-caller limits? (Many IP addresses or accounts. Real defences add overall limits and bot detection.)
5. Fail open or fail closed: which would you choose for your own system, and why?

Topic 11: Load balancing and retry at the gateway

The gateway picks one of several copies for every request, and when its pick is a dead copy that is still on the list, a Retry filter tries another copy so the visitor never sees the failure. Retry trades errors for delay.

Topic 6 showed how a copy is chosen, and Topic 7 showed that a dead copy can stay on the list for about 15 seconds. This topic joins the two.

The problem

The gateway forwards every request to one of three product copies. When a copy dies, Eureka needs about 15 seconds to notice (Topic 5), and the gateway's own saved list needs a few more seconds to refresh (Topic 4). In that window the gateway may pick a dead copy, and the visitor gets an error that was not their fault.

How the gateway picks a copy

In a route, the destination `lb://product-service` is not a real address. The prefix `lb` triggers a gateway filter that:

1. Asks the load balancer (Topic 6) for a live copy of that name, using the Eureka list (Topic 5).
2. Replaces `lb://product-service` with that copy's real address.
3. Hands the request to the HTTP client that sends it.

The lab shows this order: **Request, Route match, LoadBalancer filter, Netty HTTP client**. Netty is the engine inside the gateway that sends requests. Round-robin and random work exactly as in Topic 6. Only the place changed: now the gateway chooses for the browser (server-side, Topic 4).

The fix: a Retry filter

Retry is a per-route filter (Topic 9). If an attempt fails, the gateway chooses again. With round-robin the next choice is a different copy, so the visitor never sees the failure.

The cost: **retries turn errors into delay**. If the first copy was frozen, the gateway waited for the full timeout before trying again. In the lab the timeouts are about 2 seconds to connect and 4 seconds to wait for an answer.

Three rules about retrying

1. **Retry only safe requests.** A GET only reads, so repeating it is harmless. A POST like "create order" might have worked while the answer got lost, and a retry would create two orders. "Safe to repeat" is called **idempotent**.

2. **Limit the attempts.** Two retries is a common limit. Too many make waits longer and add load to a service that is already struggling. A burst of retries from many callers can make an outage worse, which is called a retry storm.
3. **Retry does not remove failures.** In the lab test about 23 of 24 requests succeeded, not 24 of 24.

If all three copies are down, the load balancer has nothing to choose and the gateway answers **503 Service Unavailable** at once.

Alternatives

Alternative	What to know
Nginx `proxy_next_upstream`	Can try the next server when one fails
Envoy, Istio	Retries and ejecting bad copies are built in, with no code in your service
Resilience4j (retry, circuit breaker)	A Java library for the same idea inside a program
Kubernetes Services and ingress	Load balancing is built in, and retries depend on the ingress or mesh

The natural next step beyond this lab is the **circuit breaker**: after many failures, the caller stops trying that copy for a while and fails fast.

Key words

Word	Plain meaning
`lb://`	"Choose a live copy of this service by name"
Retry	Repeat a failed request, normally on another copy
Idempotent	Doing it twice has the same effect as once, such as a read
Connect timeout	How long to wait to connect
Response timeout	How long to wait for the answer
Stale list	A list that still contains a dead copy

Word	Plain meaning
500 / 503 / 504	Program error / not available / answer took too long

In the lab

Button or area	What it does and what you should see
Round-robin, then **Send 30 requests**	Bars of 10 / 10 / 10 and a repeating colour pattern. Random gives numbers near 10 but not exact. Set it back to Round-robin afterwards
Choose **Kill an instance** and product-service-3	A line shows "Eureka currently lists UP: product-1, product-2, product-3"
Kill (crash), then immediately **Compare retry vs no retry**	Sends 24 requests down two routes that are identical except for the Retry filter and shows a table
Heal	Restarts the dead copy. Wait about 40 seconds before the next test

Test	WITH Retry	WITHOUT Retry
Kill	About 23 of 24 succeed, slightly slower on average	About 17 of 24 succeed, with errors like `500 × 7`
Freeze	Most succeed, but the average time is much higher (roughly 3 to 4 s against 2 s)	More failures or slow answers

The last box shows the gateway's saved list next to Eureka's. After a failure they disagree briefly, which explains why the gateway could still pick the dead copy.

Think about it

1. Why only retry GET requests?
2. Why not retry many times?
3. Why is the frozen case slower than the killed case?

4. How is a retry different from a circuit breaker?
5. If all three copies are down, what does the visitor see, and why?

Topic 12: API Gateway vs BFF

Different devices need different data, so one general-purpose interface forces a phone to make many calls or download too much. A Backend For Frontend (BFF) is a small helper built for one type of app, sitting behind the gateway.

This is the last topic. Topics 8 to 11 built the gateway, and this one asks whether one general front door is enough, or whether each type of app needs its own helper.

The problem

A phone app and a laptop website show the same shop, but they need different things.

- **The phone** is on a slow network with a small screen. It wants little data in as few trips as possible.
- **The laptop** has a fast connection and a big screen. It wants full details, reviews and images.

If both use the same general interface, one of two things happens:

1. The phone makes many requests, and each trip is slow on a poor network. In the lab the example is about 6 requests in two rounds, because reviews can only be requested after the product ids arrive.
2. The phone downloads data it never shows, wasting bandwidth and battery.

Two helpers, two jobs

Picture a hotel. The **main reception** at the door welcomes everyone: it checks ID, gives keys and applies house rules. That is the **API Gateway**. Business guests, families and tourists each want something different, so the hotel gives each type a **personal concierge** who prepares exactly what that guest needs in one conversation. That is a **BFF**.

BFF means **Backend For Frontend**: a small program built for one type of app. It takes one request from the device, asks the internal services many questions in parallel (fast, inside the data centre), and returns one answer shaped for that device.

	API Gateway	BFF
Purpose	Common concerns for everyone	Shaping data for one kind of app
Typical jobs	Routing, login, rate limits, logging	Combining calls, trimming fields, renaming for one screen
Nature	Mostly configuration	Real code
Owner	A platform or infrastructure team	The team that builds that app
Count	One	One per app type (web, mobile and so on)

They work together: the gateway sits in front, handles what is common, and routes to the right BFF. Keep **business rules in the real services**, not in the BFF.

In the lab, `web-bff` and `mobile-bff` are one codebase started twice with different settings. In real life they would usually be separate programs owned by separate teams.

When to use which

- **Gateway only:** all your apps need roughly the same data.
- **Add BFFs:** web and mobile need very different things, or one screen needs many calls.

The honest cost: some code is repeated between BFFs. If the same logic appears in several, move it into a shared service. You trade duplication for independence.

Alternatives

Alternative	What to know
One general API for everyone	Simplest. Fine if all apps need similar data
GraphQL	The app asks for exactly the fields it wants from one endpoint. It solves the same problem differently, and many companies use it alongside BFFs
API composition in the gateway	Some gateways can combine calls, but it can grow messy

Alternative	What to know
Federation (Apollo and others)	Merges several GraphQL services into one

Key words

Word	Plain meaning
BFF (Backend For Frontend)	A helper program built for one kind of app
Client / frontend	The app the user sees: web page, phone app
Aggregation	Collecting data from several services into one answer
Fan-out	One request causing several requests behind the scenes
Payload	The data inside a request or an answer
Round trip	One trip of a request to the server and back
Header-based routing	The gateway picks the destination by reading a header
Deployable	A program that can be started or stopped on its own

In the lab

Choose **300 ms** (a poor mobile network) and press **Load the home screen**. If you are anonymous, the lab uses alice automatically. You should see three boxes:

Box	Requests from the device	Data
1. Gateway only	About 6, in two rounds	A few KB, and the longest time
2. Gateway + web-bff	1	About 2.3 KB
3. Gateway + mobile-bff	1	About 360 bytes, about 85% smaller than web

Times change on every run, so compare ratios, not exact seconds. The 85% figure is this lab's measurement for this screen, not a general rule.

Click **last response** in a box to see the data. The web version has reviews and full details. The mobile version has only `id`, `n` (name) and `p` (price), and the short names save bytes. On the map, dots fan out from the BFF to order-service and the product shops.

Experiment	What you should see
Press `X-Client-Type: web`	Route `dsl-bff-by-header-web`, answered by web-bff
Press `X-Client-Type: mobile`	Route `dsl-bff-by-header-mobile`, answered by mobile-bff
Press no header	404, because no route matches (the predicates from Topic 8 in use)
Call both BFFs	Both answer 200
Kill (crash) web-bff, then **Call both BFFs**	The web route fails (500 or 503, red) while the mobile route still works (green). Press **Heal** afterwards

Think about it

1. Isn't a BFF just another gateway? How are they different?
2. Why not build one big API for every app?
3. Why is it good that a failure in web-bff does not affect mobile?
4. Where should login live: the gateway or each BFF?
5. When is GraphQL a better choice than BFFs?
6. Why should business rules stay out of the BFF?

Wrap-up: how it all fits together

A request travels through the whole system, and each topic protects one part of the journey. A visitor reaches the **gateway** (Topic 8), which checks identity and stamps a request id (Topic 9) and applies a speed limit kept in Redis (Topic 10). It then looks up a live copy by name in **Eureka** (Topics 4 and 5), chooses one with a **load balancer** (Topics 6 and 11), and retries if the pick was dead (Topic 11). Phone and web visitors may be handed to

their own **BFF** first (Topic 12). Meanwhile every program reports its health and numbers (Topics 2 and 7), **Prometheus** and **Grafana** collect and chart them (Topic 3), and **ELK** gathers every log line so one request id tells the whole story (Topic 1).

One sentence per topic

Topic	The sentence to remember
1. Logging (ELK)	Put every program's diary in one searchable place, and give each request a code to follow it.
2. Actuator and Micrometer	Every program can answer "how are you?" and "give me your numbers" over the web.
3. Prometheus and Grafana	One clerk collects the numbers on a timer, one board draws them, and an alarm rings when something is wrong.
4. Discovery patterns	Find services by name. Either the caller chooses (client-side) or a middle program chooses (server-side).
5. Eureka	A directory kept alive by registering, heartbeats and goodbyes, which refuses to remove names during a storm.
6. Load balancing	Take turns (round-robin) or roll a die (random) to share work between identical copies.
7. Health checks	A program can be unwell, polite, crashed or frozen, and the system finds out in different ways and at different speeds.
8. Gateway routes	One door with a table of rules. The first rule that fits wins, and no match is 404.
9. Gateway filters	Do identity checks, logging and rewriting once at the door: some for everyone, some for one route.
10. Rate limiting	A bucket of tickets per visitor, kept in Redis. No ticket means 429.
11. Gateway load balancing	The gateway shares work and retries on another copy when one fails, trading errors for delay.

Topic	The sentence to remember
12. API Gateway vs BFF	One door for everyone, plus one personal helper for each kind of app.

Ideas that repeat across topics

- **Tools change, concepts stay.** Every topic lists alternatives. Learn the concept first.
- **Alive is not healthy.** A heartbeat proves a program can send a message, not that it can do its job.
- **There is always a gap.** Between a failure and its detection, callers can be sent to a dead program. That is why timeouts and retries exist.
- **Slow failures are worse than fast ones.** A frozen program is worse than a crashed one.
- **Availability versus accuracy.** Self-preservation and fail-open both choose to keep the system running at the cost of occasional wrong answers.
- **Do shared jobs once.** Identity, logging and limits belong at the gateway, and business rules belong in the services.
- **Lab shortcuts are not production.** The lab shortens Eureka's timers about 10 times, uses simple tokens, and lets a header choose the plan.

Self-check

Try to answer these without looking back.

1. A customer says an order failed at 2:15. Which tools do you use, and in what order?
2. Why does Prometheus pull while Eureka relies on programs reporting in?
3. What is the difference between client-side and server-side discovery? Give the lab's example of each.
4. A product copy crashes. List what happens in the first 15 seconds, and who is hurt.
5. Why can Eureka keep dead programs listed, and is that a good thing?
6. Why must the gateway remove the identity headers a visitor sends?
7. What does a bucket of size 5 refilling at 2 per second allow, and what happens to a burst of 10?
8. Why retry only GET requests?
9. When would you add a BFF, and when is the gateway alone enough?

10. For any tool in this guide, name one alternative and one reason a company might pick it instead.